🚀 Using Jekyll for Static Site Generation

🤔 What is Jekyll?

Jekyll is a **static site generator** that converts plain text files written in Markdown or HTML into a complete static website. It's widely used for blogs, portfolios, and project sites, especially because it integrates seamlessly with GitHub Pages for free and easy hosting.

💡 Why Use Jekyll?

- ✍️ **Simple authoring:** Write content in Markdown or HTML without needing a backend server.
- ⚡ **Fast loading:** Static sites load quickly since they don't depend on server-side processing.
- 🎨 **Highly customizable:** Templates, layouts, and front matter let users control design and URLs.
- 🌐 **GitHub Pages ready:** Automatically build and host sites for free on GitHub.
- 🧑‍🤝‍🧑 **Active community:** Many plugins, themes, and resources available.

🛠️ How to Install Jekyll

Step 1: Install Ruby

Jekyll requires Ruby. The user must install Ruby appropriate for their OS:

- **Windows:** Download Ruby+Devkit from rubyinstaller.org
- **macOS:** Ruby usually comes pre-installed. Optionally upgrade via Homebrew:

```
brew install ruby
```

- **Linux:** Use the package manager, for example:

```
sudo apt-get install ruby-full build-essential
```

Step 2: Install Bundler and Jekyll Gems

In the terminal or command prompt, run:

```
gem install bundler jekyll
```

> *Note: On Windows, the terminal may need to be run as Administrator.*

Step 3: Verify Installation

Verify Jekyll is installed correctly by running:

```
jekyll -v
```

The installed Jekyll version should be displayed.

📂 Understanding the Core Jekyll Concepts**

Before working with Jekyll, it helps to understand a few important files, folders, and concepts that control how the website is organized and generated.

📝 1. YAML Front Matter

YAML front matter is a block of metadata placed at the very beginning of a Markdown or HTML file, enclosed between three dashes (`---`). It defines page-specific settings such as the title, layout, permalink, and publication date.

```
---
title: My First Blog
layout: default
permalink: /my-first-blog
---
```

- **`title`** — Defines the page or blog post title when used by the templates.
- **`layout`** — Specifies the layout template used to render the page.
- **`permalink`** — Defines the page's output URL.
- **`date`** — Can specify the publication date of a post.

🔗 2. Permalinks

A permalink determines the URL at which a generated page will be accessible. For example:

```
---
permalink: /about
---
```

This makes the page available at `/about` on the website. Permalinks help create clean, readable URLs instead of relying on default file paths.

⚙️ 3. The `_config.yml` File

The `_config.yml` file stores the website's global configuration, including the site title, URL, base URL, default layouts, and other settings.

```
title: Madhurima Mindscape
url: "https://madhurimarawat.github.io"
baseurl: "/Madhurima-Mindscape"
```

- **`title`** — Sets the site's name.
- **`url`** — Defines the website's main URL.
- **`baseurl`** — Specifies the repository path when hosting a project site on GitHub Pages.

Jekyll reads this configuration when building the website, making these settings available to templates through variables such as `site.title` and `site.baseurl`.

📁 4. The `_site` Folder

The `_site` folder contains the generated static website created when Jekyll builds the project. It includes the processed HTML pages, copied assets, and other output files.

- **Source files —** the original Markdown, HTML, CSS, and other project files.
- **Generated files —** The final files produced by Jekyll inside `_site`.

We can generate the site using:

```
jekyll build
```

To preview it locally, run:

```
jekyll serve
```

Jekyll generates the site into `_site` by default. This folder is build output, so you generally should not edit its contents directly.

🪄 5. Liquid Templates and Variables

Liquid is the templating language Jekyll uses to insert dynamic values and control template logic. It lets us reuse site configuration, page metadata, and content.

For example:

```
{{ "{ site.title " }}}
```

A page can access its own metadata using `page` variables, while global configuration is available through `site` variables. Liquid also supports loops, conditions, and includes, making it easier to build reusable layouts.

💡 **Quick Tip:** YAML front matter defines page-specific metadata, `_config.yml` stores global settings, `_layouts/` and `_includes/` help reuse HTML, and `_site/` contains the generated website. Understanding these concepts makes it much easier to create, organize, and maintain a Jekyll-powered website.

📂 How to Use This Project with Jekyll

1. 🔗 **Clone the repository**:

   ```
   git clone https://github.com/madhurimarawat/Madhurima-Mindscape.git
   ```

2. 📁 **Navigate into the project folder**:

   ```
   cd Madhurima-Mindscape
   ```

3. 📝 **Ensure all files have proper front matter** — YAML metadata at the top of each page, defining permalinks, titles, etc.

4. 💻 **Run the local Jekyll server** to build and preview the site:

   ```
   jekyll serve
   ```

5. 🌐 **Open a browser and go to**:

   ```
   http://localhost:4000
   ```

6. 🔄 **Preview the site live** — changes to files auto-refresh the browser.

▶️ How to Test Jekyll Locally

- Open a terminal and navigate to the project folder.
- Run:

  ```
  jekyll serve
  ```

- Wait for the build to complete.
- Visit `http://localhost:4000` in a browser.
- Modify content or layout files and watch the live reload.

✅ Summary

Jekyll is a powerful tool for creating static websites that are fast, easy to maintain, and perfect for personal or project portfolios. By installing Ruby and Jekyll, cloning the project repository, and running the local server, anyone can preview and develop the site before publishing it live, especially on GitHub Pages.